

Bitwise operations

A	B	and $A \wedge B$	or $A \vee B$	not $\neg A$	xor $A \oplus B$
F ⁰	F ⁰	F ⁰	F ^x	T	F ⁰
F ⁰	T ¹	F ⁰	T ^x	T	T ¹
T ¹	F ⁰	F ^x	T ¹	F	T ¹
T ¹	T ¹	T	T	F	F ⁰

$\left. \begin{matrix} \neq T \\ = F \end{matrix} \right\} \xrightarrow{\text{xor}}$

`; AL = 1111 0000b`

`and AL, 001111 00b`

`; AL = 0011 0000b`

→ I am isolating a few bits.

or

→ I am combining a few bits

xor

→ I am complementing a few bits.

shl } faster than mul, div.
shr }

rol } real-life example: msb → combine upper part, lower part.
ror } (include the carry flag, etc.)

1. EXPRESSIONS = sth that uses an operand and operators.

1.1. OPERANDS

immediate (ct) → we can determine it at assembly time.
→ ex: 2, 5, ...

register → at run time

memory → at loading time

selector.
↑
memory → beginn. of segment → far address

How to declare an immediate operand:

hex: "b" / "x"

- binary: "b" / "y"
- dec: "d" / "t"
- octal: "g" / "o"

↓
suffix

Examples:

0xB2A
 0xb2a
 0h b2A
 0B2ah
 0b2AH
~~B2Ah~~

} → hex, B2A
 (not case sensitive!)

$$\left. \begin{array}{l} 0y1100 - 1000 \\ 1100 - 1000y \\ 11001000b \end{array} \right\} \rightarrow \text{bin}, 11001000$$

?

```
mov    eax, 8
```

```
mov eax, [v];
```

$\swarrow$

mov eax, DWORD PTR [DS: 004027AB] $\Rightarrow$ 00YDBG.

$\downarrow \qquad\qquad\qquad \downarrow \qquad\qquad\qquad \downarrow$

non-destructive data seg. actual
conversion ↙ offset

How to declare register operands:

mov eax, ebx indirect addressing, usage

How to declare memory operands:

offset formula = $[base] + [index * scale] + [ct.]$

- offset: - when is that value computed? → at assembly time
- offset of a **direct addressing operand**
- search for that value in memory.

CS, DS, ES

ES: [var] ← DIRECT ADDRESSING MODE

↓

mov dh, [edx + ecx * 4 + 3] ← INDIRECT ADDRESSING MODE

1.2. OPERATORS

- perform computations with constant scalar values

↓
assembly time.

PRIORITY	OPERATOR	TYPE	RESULT
7	-	<u>unary</u> , prefix	$-x = 0 - x$ (2's comp. of x) mov eax, -2 ✓
7	+	<u>unary</u> , prefix	$+x = x$ (no effect)
7	~	<u>unary</u> , prefix	(1's comp.) mov al, 0; al = 0FFh
7	!	<u>unary</u> , prefix	$!x = 0$ if $x \neq 0$, else 1 (logic negation)
6	*	binary, infix (2) (between)	$1 * 2 * 3 = 6$ (multiplication) mov eax, 1 * 2 * 3; eax = 6
6	/	binary, infix	$24 / 4 / 2 = 3$ (unsigned division) quotient
6	//	binary, infix	$-24 / 4 / 2 = -3$ (signed division)
6	%	binary, infix	remainder (unsigned division)
6	%%	binary, infix	remainder (signed division) $-123 \% \% 100 \% \% 5 = -3$.
5	+	binary, infix	Sum : $1 + 2 = 3$
5	-	binary, infix	subtraction
4	<<	11-11-	bitwise left shift $1 << 4 = 16$ (2^4)
4	>>	11-11-	bitwise right shift $1 >> 4 =$
3	&	11-11-	AND
2	^	11-11-	(+)
1		11-11-	OR

Computed first

Computed last

Order of operations:

5 | 6 + 7 & 8
(3) (1) (2)

= 5 | ((6 + 7) & 8)

= 5 | (13 & 8)

= 5 | 8

= 0Dh

unary ↗ higher priority

binary ↘ lower priority

1.2.1. TYPE OPERATORS

- non destructive conversions to a variable.

BYTE, WORD, DWORD, QWORD : data size specs.

mov [mem], 12 syntax error (operation size not specified)

↓
not an error, but this would be an error:

because "bits32"
tells CPU to
work on 32 bits

↓
converts
automatically

data seg....
string_1 resb 20

;
code seg...

mov [string_1 + 2], 12 ^{byte} ✓

When should we specify the type?

div ^{type} ✓ [mem]

mul ^{type} ✓ [mem]

push ^{type} ✓ [mem]

pop ^{type} ✓ [mem]

but [push 15 ✓ (implicit type) (=) push dword 15.]

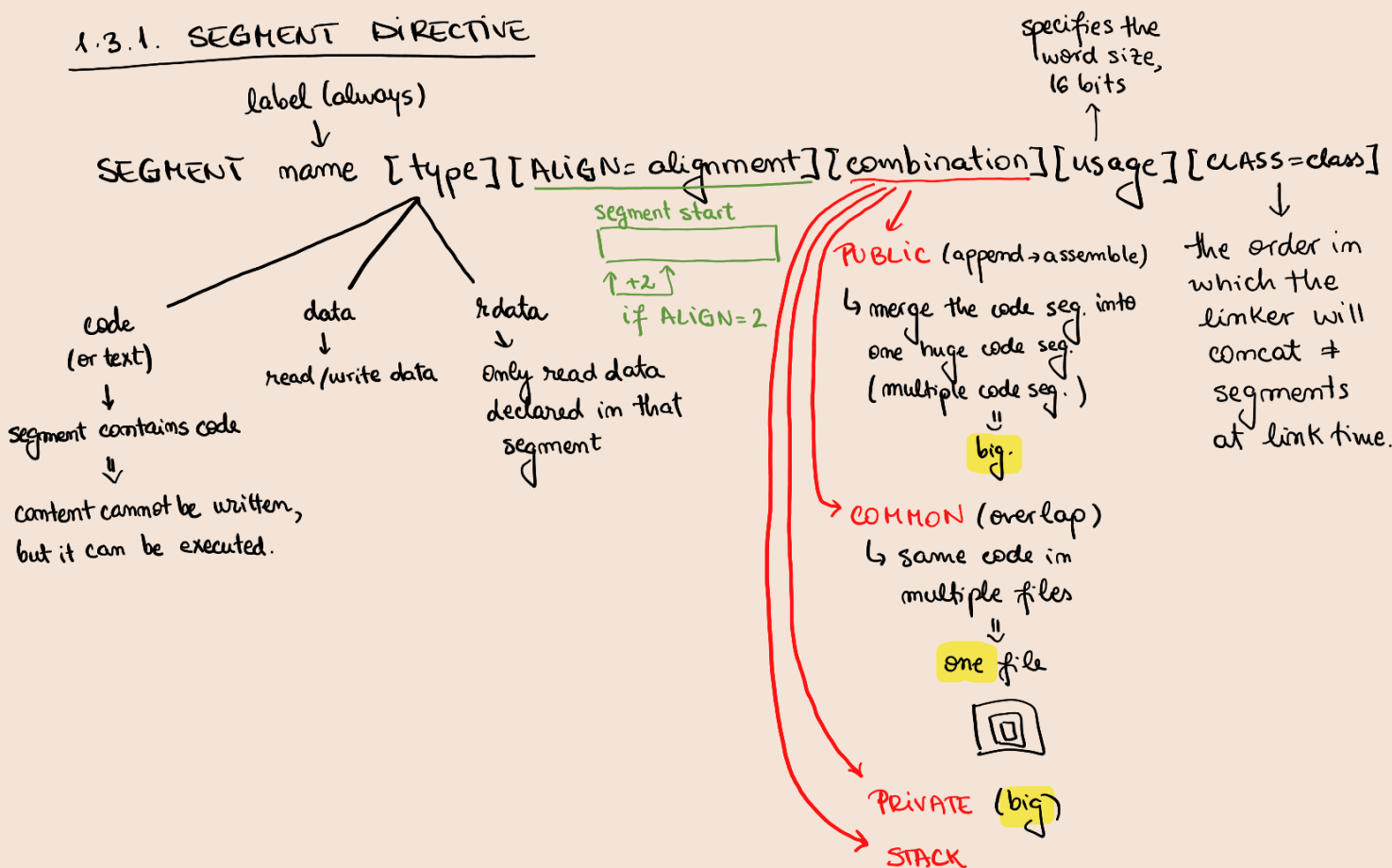
1.2.2. IMPLICIT OPERANDS ON 64 BITS.

div dword [mem]

↳ take the **qword** represented on EDX:EAX, divide by mem.

1.3. DIRECTIVES

1.3.1. SEGMENT DIRECTIVE



Example: segment code use32 class=code:

1.3.2. DATA DIRECTIVE

data def = declaration + allocation

↓ ↓ ↓
unique. not unique. unique.

because I can declare data in multiple ways.

How can I declare data?

data type = size of representation : byte / word / dword / qword

General form of a data declaration:

How can I declare a matrix?

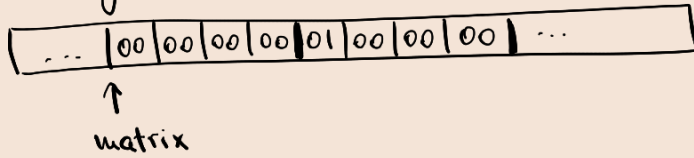
matrix DD 0,1,4,9,16,25

DD 49,64,...

DD 100,121,...

→ how to access? +6 (DD)

data seg.



Uninitialised data reservation

RES X

↳ B/W/D/Q/T

- just reserve memory → fill eventually.

ex: table1 RESW 100h

↳ 256 words reserved starting from offset table1 in the data segment.

1.3.3. TIMES DIRECTIVE

TIMES factor data-type expression

→ assign a value.

ex: matrix2 times 10*10 dd 0.

times 32 add eax, ebx (repeat 32 times the value stored in ebx, eax).

1.3.4. EQU

(name EQU expression)

→ declare constant

ex: end-of-data EQU '!'
buff-size EQU 1000h

} these constants won't appear in the data segment (array)

data seg

a db 2

b equ 3 → will not appear in the array → 32 bits (later.)

c db 4